Контест 11 · Деревья игр и перебор

Неделя 22 · темы недель 21–22: перебор с возвратом и поиск в ширину, минимакс и альфа-бета, выигрышные позиции, ускорения перебора; в проектной части — ИИ-соперник и нагрузка. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. Перебор

A1. `subset_sums(nums, target)` — сколько подмножеств (выбираем позиции, одинаковые числа на разных местах различаются) дают сумму `target`. Пустое подмножество — сумма 0. Сколько всего подмножеств проверит перебор?

```
Ввод:  subset_sums([3, 1, 4, 1, 5], 5), subset_sums([2, 2, 2], 4), subset_sums([], 0)
Вывод: 4, 3, 1
```

A2. `n_queens(n)` — сколько способов поставить `n` ферзей на доску `n × n`, чтобы никто никого не бил. Перебор с возвратом: по ферзю в строке, занятые столбцы и диагонали — в множествах.

```
Ввод:  [n_queens(n) for n in range(1, 9)]
Вывод: [1, 0, 0, 2, 10, 4, 40, 92]
```

A3. `knight_distance(a, b)` — наименьшее число ходов коня с клетки `a` на клетку `b` доски 8×8 (`"a1"` … `"h8"`). Поиск в ширину.

```
Ввод:  knight_distance("a1", "b3"), knight_distance("a1", "h8")
       knight_distance("a1", "b2")
Вывод: 1, 6
       4
```

A4*. `sliding_2x3(start)` — пятнашки 2×3: строка из 6 символов построчно, `"_"` — пустая клетка. Цель — `"12345_"`. Ход — сдвинуть в пустую клетку соседнюю по стороне фишку. Наименьшее число ходов или `-1`, если собрать нельзя. Почему ровно половину позиций собрать нельзя?

```
Ввод:  sliding_2x3("1234_5"), sliding_2x3("_12345"), sliding_2x3("21345_")
Вывод: 1, 15, -1
```

B. Деревья игр

B1. `nim(heaps)` — ним: ход — взять сколько угодно (хотя бы один) камней из одной кучки, кто не может ходить — проиграл. Ответ — `("second", None)`, если ходящий проигрывает, иначе `("first", (кучка, сколько оставить))` — выигрывающий ход в кучке с наименьшим номером. Подсказка: XOR размеров.

```
Ввод:  nim([3, 4, 5]), nim([1, 2, 3]), nim([7])
Вывод: ("first", (0, 1)), ("second", None), ("first", (0, 0))
```

B2. `game_value(tree, maximizing=True)` — дерево игры: вложенные списки, лист — число. В корне ходит максимизирующий, на следующем уровне — минимизирующий, и так по очереди.

```
Ввод:  game_value([[3, 5], [2, 9], [0, 1]])
       game_value([[[1, 8], [3, 2]], [[9, 4], [6, 7]]])
Вывод: 3
       7
```

B3. `alphabeta_leaves(tree)` — какие листья посмотрит альфа-бета (дети слева направо, отсечение при `alpha ≥ beta`) — по порядку просмотра. Нарисуйте дерево из примера и отметьте отсечённые ветки.

```
Ввод:  alphabeta_leaves([[3, 5], [2, 9], [0, 1]])
       alphabeta_leaves([[5, 6], [7, 4, 5], [3, 1]])
Вывод: [3, 5, 2, 0]
       [5, 6, 7, 4, 3]
```

C. Выигрышные позиции

C1. `ttt_games()` — сколько разных партий крестиков-ноликов: последовательностей ходов до победы или до заполнения поля. Ответ — `{"total", "x", "o", "draw"}`.

```
Ввод:  ttt_games()
Вывод: {"total": 255168, "x": 131184, "o": 77904, "draw": 46080}
```

C2. `losing_positions(n, moves)` — игра «бери камни»: за ход берём `k ∈ moves` камней (если хватает), кто не может ходить — проиграл. Все позиции `0..n`, проигрышные для того, кто ходит. Позиция проигрышная, если любой ход ведёт в выигрышную.

```
Ввод:  losing_positions(20, {1, 3, 4}), losing_positions(10, {1, 2})
Вывод: [0, 2, 7, 9, 14, 16], [0, 3, 6, 9]
```

C3. `grundy(n, moves)` — числа Шпрага — Гранди той же игры: `g(s) = mex {g(s − k)}`, `mex` — наименьшее неотрицательное число не из множества. `sum_winner(heaps, moves)` — несколько кучек, ход — в одной кучке по правилам C2: `"first"`, если XOR чисел Гранди кучек не ноль, иначе `"second"`.

```
Ввод:  grundy(10, {1, 3, 4})
       sum_winner([2, 7], {1, 3, 4}), sum_winner([3, 5, 6], {1, 2})
Вывод: [0, 1, 0, 1, 2, 3, 2, 0, 1, 0, 1]
       "second", "first"
```

C4*. `principal_variation(tree)` — главная линия: номера детей на пути от корня до листа, если оба играют идеально (дерево как в B2); при равных значениях — левый ребёнок. Чем она полезна боту, который думает над следующим ходом?

```
Ввод:  principal_variation([[3, 5], [2, 9], [0, 1]])
       principal_variation([[[1, 8], [3, 2]], [[9, 4], [6, 7]]])
Вывод: [0, 0]
       [1, 1, 1]
```

D. Ускорения

D1. `grid_paths(grid)` — число путей из левого верхнего угла в правый нижний ходами вправо и вниз; `"#"` — стена. Перебор с запоминанием (`functools.cache`): поле 16×16 без стен — мгновенно.

```
Ввод:  grid_paths(["...", "...", "..."]), grid_paths(["..#", "...", "#.."])
       grid_paths(["." * 16] * 16)
Вывод: 6, 4
       155117520
```

D2. Хэширование Зобриста — ключ таблицы транспозиций. `zobrist_table(cells, pieces, seed=58)`: `rng = random.Random(seed)`, `table[клетка][фигура] = rng.getrandbits(64)` (клетки снаружи, фигуры внутри). `zobrist(board, table)` — XOR `table[i][v − 1]` по клеткам с фигурой `v ≥ 1`. `zobrist_move(h, cell, old, new, table)` — хэш после замены `old` на `new` в одной клетке за O(1).

```
Ввод:  t = zobrist_table(9, 2); h = zobrist([1, 0, 0, 0, 2, 0, 0, 0, 0], t); hex(h)
       zobrist([1, 0, 0, 0, 2, 0, 0, 0, 1], t) == zobrist_move(h, 8, 0, 1, t)
Вывод: "0x414f0e3a63d72d85"
       True
```

D3. `order_gain(tree)` — сколько листьев посмотрит альфа-бета (B3) в исходном порядке и после идеального упорядочения: у каждого узла дети отсортированы по значению (B2) — лучшие для ходящего первыми, при равенстве порядок сохраняется. Ответ — пара чисел.

```
Ввод:  order_gain([[[1, 8], [3, 2]], [[9, 4], [6, 7]]])
       order_gain([[3, 5], [2, 9], [0, 1]])
Вывод: (8, 5)
       (4, 4)
```

E. Проект и алгоритмы

E1. ИИ-соперник вашей игры (семинар 21): три позиции с известным ответом («выиграй за ход», «заблокируй», «не попадись в вилку»), тесты на них и лимит времени на ход.

E2. Отчёт о нагрузке (семинар 22): таблица «до/после» по p95 и ошибкам, одна оптимизация агента, которая помогла, и одна, которая не дала ничего.

E3. Две реплики API за Nginx: игроки одной комнаты на разных репликах видят ходы друг друга (Redis pub/sub). Покажите, где хранится состояние комнаты.

E4 · босс. Кубик в дереве игры. Ответы — `fractions.Fraction`. За каждую часть свой плюс.

а) `expectimax(node)` — узел: число (лист) или пара `(тип, дети)`, тип `"max"`, `"min"` или `"chance"`; у `"chance"` дети — пары `(вероятность, узел)`, сумма вероятностей не 1 — `ValueError`. Значение случайного узла — среднее, взвешенное вероятностями.

```
Ввод:  h = Fraction(1, 2)
       expectimax(("max", [("chance", [(h, 10), (h, 0)]), ("min", [4, 7])]))
Вывод: Fraction(5, 1)
```

б) `reach_probability(target)` — игра «Свинья»: бросаем кубик; выпала 1 — ход сгорает, 2–6 — прибавляются к сумме хода. Бросаем, пока сумма меньше `target`. Вероятность набрать `≥ target`.

```
Ввод:  reach_probability(2), reach_probability(20)
Вывод: Fraction(5, 6), Fraction(5675645, 15116544)
```

в) `expected_hold(k)` — ожидаемые очки за ход при стратегии «бросать, пока сумма меньше `k`»; `best_hold(limit=100)` — все `k` из `1..limit` с наибольшим ожиданием, по возрастанию.

```
Ввод:  expected_hold(2), best_hold()
Вывод: Fraction(10, 3), [20, 21]
```

E5*. `solve_connect(cols, rows, k)` — кто выигрывает «k в ряд» (фишки падают вниз, первый ходит в любой столбец) при идеальной игре: `1`, `-1` или `0`. Нужны таблица транспозиций и ранний выход по найденной победе; `(4, 4, 4)` должно решаться за секунды. Что изменится, если проверять победу только у последней фишки?

```
Ввод:  solve_connect(3, 3, 3), solve_connect(4, 3, 3), solve_connect(4, 4, 4)
Вывод: 0, 1, 0
```